

****Contribuer Efficacement******Bonnes pratiques**

:::

Session 7 – Cours M1 Introduction aux Logiciels Libres

****Objectifs de la séance****

****Ce que vous saurez faire****

À la fin de cette séance, vous serez capables de :

- **Rédiger** une pull request de qualité
- **Écrire** des messages de commit clairs et normalisés
- **Communiquer** efficacement avec les mainteneurs
- **Participer** à la code review (recevoir et donner)
- **Lancer** votre propre projet open source
- **Gérer** les versions et les releases d'un projet

****Partie 1****

L'anatomie d'une bonne PR

****Qu'est-ce qu'une bonne PR ?******Caractéristiques**

- Un seul changement logique
- Taille raisonnable (< 500 lignes)
- Bien documentée
- Testée
- Conforme aux standards

Objectif

- Faciliter la review
- Minimiser les risques
- Accélérer le merge
- Respecter le temps des reviewers

****Structure d'une PR****

Titre : Court, descriptif, au présent

-  "Fix memory leak in image parser"
-  "Fixed stuff"

Description :

```
\## What does this PR do?  
[Description claire du changement]
```

```
\## Why is this change needed?  
[Contexte, référence à l'issue]
```

```
\## How to test?  
[Instructions pour tester]
```

```
\## Screenshots (if applicable)  
[Captures d'écran]
```

****Bonnes pratiques pour les PRs****

1. **Une PR = Un changement** – Pas de PR "fourre-tout"
2. **Référez l'issue** – "Fixes #123" ou "Closes #456"
3. **Petites PRs > Grosses PRs** – Plus faciles à reviewer
4. **Expliquez le "pourquoi"** – Le code montre le "quoi"
5. **Auto-review avant de soumettre** – Relisez votre diff
6. **Tests inclus** – Prouvez que ça marche

****Ce qui ralentit une PR******Problèmes courants**

- PR trop grosse
- Pas de description
- Changements non liés mélangés
- Tests manquants
- Conflits avec main

Solutions

- Découper en plusieurs PRs
- Utiliser le template
- Une PR par sujet
- Ajouter les tests
- Rebaser régulièrement

****Partie 2****

Messages de commit

****Pourquoi c'est important****

- **Historique lisible** – Comprendre l'évolution du projet
- **Debugging** – **git bisect**, **git blame**
- **Changelog automatique** – Release notes
- **Communication** – Avec l'équipe, avec votre futur vous

****Structure d'un message de commit****

```
<type>(<scope>): <subject>  
<ligne vide>  
<body>  
<ligne vide>  
<footer>
```

Exemple :

```
fix(parser): handle empty input gracefully
```

Previously, passing an empty string would cause a crash.
Now it returns an empty result.

Fixes #42

****Conventional Commits****

Standard de plus en plus adopté.

Type	Usage
feat	Nouvelle fonctionnalité
fix	Correction de bug
docs	Documentation uniquement
style	Formatage, pas de changement de code
refactor	Refactoring sans changement de comportement
test	Ajout/modification de tests
chore	Maintenance, dépendances, CI

<https://www.conventionalcommits.org>

****Règles d'or des messages****

1. **Première ligne < 50 caractères**
2. **Utiliser l'impératif** : "Add" pas "Added" ou "Adds"
3. **Pas de point à la fin** du sujet
4. **Corps à 72 caractères** par ligne
5. **Expliquer le "pourquoi"** dans le corps
6. **Référencer les issues** dans le footer

****Exemples : Mauvais vs Bon******Mauvais** ❌

fix bug

WIP

changes

Updated the thing and also
fixed that other thing and
refactored some stuff

Bon ✅

fix(auth): prevent session timeout
during file upload

feat(api): add rate limiting to
public endpoints

docs: update installation
instructions for Windows

****Partie 3******Communication**

****La communication asynchrone****

L'open source fonctionne souvent en **asynchrone** :

- Fuseaux horaires différents
- Contributeurs bénévoles
- Temps de réponse variable

Implications :

- Soyez clair et complet dès le premier message
- N'attendez pas de réponse immédiate
- Évitez les "ping" trop fréquents

****Règles de communication****

À faire

- Être poli et respectueux
- Fournir le contexte nécessaire
- Poser des questions claires
- Accepter le feedback
- Remercier les reviewers

À éviter

- Ton agressif ou impatient
- "Ça marche pas" (sans détails)
- Relancer trop vite
- Prendre les critiques personnellement
- Argumenter indéfiniment

****Anti-patterns de communication******"Don't ask to ask" ❌**

"Quelqu'un peut m'aider ?"

→ Posez directement votre question avec le contexte.

"XY Problem" ❌

"Comment je fais X ?" (alors que vous voulez Y)

→ Expliquez votre objectif réel.

"Help vampire" ❌

Demander de l'aide sans effort préalable

→ Montrez que vous avez cherché d'abord.

****Comment poser une bonne question****

\## Ce que j'essaie de faire
[Objectif]

\## Ce que j'ai essayé
[Vos tentatives]

\## Ce qui se passe
[Comportement actuel, erreurs]

\## Ce que j'attendais
[Comportement souhaité]

\## Environnement
[Versions, OS, etc.]

****Partie 4******Code Review**

****Recevoir une review****

La code review n'est **pas une attaque personnelle**.

Mindset :

- Le reviewer essaie d'améliorer le projet
- Vous apprenez de chaque review
- Les mainteneurs ont une vision globale
- Un refus n'est pas un échec

****Types de commentaires de review****

Type	Signification	Réaction
Suggestion	"Consider doing X"	Évaluer, appliquer si pertinent
Question	"Why did you ...?"	Expliquer votre raisonnement
Nitpick	Détail mineur (nit:)	Corriger sans discuter
Bloquant	"This must be changed"	Obligatoire avant merge
Praise	"Nice approach!"	Dire merci

****Répondre aux commentaires****

Pour chaque commentaire :

1. Lire attentivement
2. Comprendre le point soulevé
3. Répondre OU corriger OU discuter

Exemples de réponses :

- "Good point, fixed in abc123"
- "I chose this approach because ... What do you think?"
- "I see your point, but X because Y. Happy to change if you prefer."

****La review comme contribution****

La code review n'est **pas réservée aux mainteneurs**. Vous pouvez contribuer en reviewant les PRs des autres, même sans droits de commit.

Dans le projet Subversion, Greg Stein s'est mis à reviewer **chaque commit** qui entrerait dans le dépôt. Il n'a jamais demandé aux autres de faire pareil – il a simplement montré l'exemple. Rapidement, d'autres contributeurs ont commencé à faire de même. – Karl Fogel, *Producing Open Source Software*

Avant de reviewer :

- Vérifier que le projet **accueille** les reviews externes
- Comprendre les **attentes** du projet pour les reviewers

****Donner une review****

Si vous êtes reviewer :

Soyez constructif

- Expliquer le "pourquoi"
- Proposer des alternatives
- Reconnaître le bon travail
- Distinguer bloquant vs suggestion

Évitez

- Commentaires vagues
- Ton condescendant
- Bikeshedding (détails infinis)
- Bloquer sans explication

****Partie 5****

Pièges à éviter

****Erreurs fréquentes****

1. **Commencer à coder sans discuter** – Sur les grosses features, discutez d'abord
2. **PR massive** – Impossible à reviewer, souvent abandonnée
3. **Ignorer les guidelines** – Lire CONTRIBUTING.md !
4. **Push force sans prévenir** – Peut casser la review en cours
5. **Disparaître après la PR** – Répondez aux commentaires

****Avant de commencer à coder******Étapes essentielles :**

1. **Vérifier que l'issue n'est pas prise**
 - Quelqu'un travaille-t-il déjà dessus ?
 - Y a-t-il une PR en cours ou abandonnée ?
2. **Signaler votre intention**
 - Commentez l'issue : "I'd like to work on this"
 - Attendez une confirmation du mainteneur si nécessaire
3. **Synchroniser votre fork**
 - **git fetch upstream && git rebase upstream/main**
 - Faire ça **avant** de créer votre branche
4. **Comprendre le contexte**
 - Lire les discussions sur l'issue
 - Vérifier les PRs liées ou fermées

****Garder son fork à jour******Pourquoi c'est important :**

- Évite les conflits de merge massifs
- Votre code est basé sur la dernière version
- Les tests passent sur le code actuel

Quand synchroniser :

Moment	Obligatoire ?
Avant de créer une branche	✓ Oui
Régulièrement pendant le travail	Recommandé
Avant de soumettre la PR	✓ Oui
Quand le reviewer le demande	✓ Oui

```
git fetch upstream  
git rebase upstream/main
```

****Le Signed-off-by et DCO****

Certains projets (Linux, CNCF) exigent un DCO.

```
git commit -s -m "Add new feature"
```

Ajoute automatiquement :

```
Signed-off-by: Your Name <your@email.com>
```

Signification : "Je certifie que j'ai le droit de soumettre ce code."

****Gérer le rebase****

Votre PR a des conflits avec main ?

```
\# Récupérer les derniers changements  
git fetch upstream  
  
\# Rebaser votre branche  
git rebase upstream/main  
  
\# Résoudre les conflits si nécessaire  
\# ... éditer les fichiers ...  
git add .  
git rebase --continue  
  
\# Force push (avec précaution)  
git push --force-with-lease origin ma-branche
```

****Partie 6****

Démarrer son propre projet

****Quand ouvrir son code ?******Contextes courants**

- Application personnelle ou académique
- Bibliothèque réutilisable
- Outil non-cœur-de-métier de votre entreprise
- Logiciel pour un client sans besoin d'exclusivité
- Produit principal (modèle Red Hat)

Motivations

- Partager (générosité)
- Recevoir des contributions
- Rendre le code auditable
- Attirer des développeurs / recruter
- Créer un standard commun

****Ouvrir le code dès le début****

■ "Le projet ne sera **jamais** suffisamment prêt."

Pourquoi ne pas attendre ?

- Plus on attend, plus le **nettoyage** est lourd
- Vous pourriez recevoir des contributions **plus tôt que prévu**
- Ouvrir le code **n'oblige pas** à :
 - écrire de la documentation exhaustive
 - être disponible pour répondre aux questions
 - accepter toutes les contributions

Avant de recevoir des contributions → choisir une licence.

****Checklist de lancement****

Étape	Détail
Choisir une licence	Commencer permissif (MIT) si on hésite ; on peut toujours aller vers plus restrictif
Créer le dépôt	Sur une forge (GitHub, GitLab, Codeberg...)
README.md	Mission en 1 phrase, résumé en 1 paragraphe, installation, usage
LICENSE	Fichier avec le texte complet de la licence
CONTRIBUTING.md	Comment contribuer, standards de code, processus
.gitignore	Adapté au langage / framework
CI/CD	Au minimum : lancer les tests automatiquement

****Choix technologiques****

Règle clé : réduire les besoins d'apprentissage.

Moins de choses à apprendre = plus de contributeurs potentiels, plus vite productifs.

En pratique :

- Langage, framework, outils de build : **standards** de l'écosystème
- Éviter les solutions "maison" sans justification
- Utiliser les outils que la communauté cible connaît déjà

****Documentation progressive****

Ne pas essayer de tout documenter d'un coup. Procéder par **niveaux** :

1. **Mission en une phrase** – "X is a Y that does Z"
2. **Résumé en un paragraphe** – Problème résolu, public cible
3. **Quick start** – Installation et premier usage en 5 minutes
4. **Exemples** – Cas d'usage concrets, démo, captures d'écran
5. **Référence** – API, configuration, options
6. **Guide du développeur** – Architecture, comment contribuer au code

La documentation utilisateur est **essentielle**. La documentation développeur est **idéale**.

****Annoncer le projet****

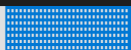
Pas besoin d'attendre que tout soit parfait.

Progressivement

- 1 Parler à des connaissances intéressées
- 2 Répondre à des besoins exprimés sur des forums
- 3 Annonce officielle lors de la première version

Canaux d'annonce

- Mailing lists spécialisées
- Reddit, Hacker News, Lobsters
- Forums du langage / framework
- Réseaux sociaux (Mastodon, Twitter/X)
- Conférences, meetups

****Partie 7******Gestion des versions (Release Management)**

****Pourquoi produire des versions ?****

Le code est déjà sur un dépôt public. Alors pourquoi des versions numérotées ?

- Désigner une version **recommandée** pour les utilisateurs
- Faire connaître les **changements** récents
- Distribuer via les **gestionnaires de paquets** (npm, pip, apt...)
- Produire des **installateurs** / exécutables
- Permettre aux utilisateurs de décider **quand mettre à jour**

****Versionnement sémantique (semver)****

Format : **MAJEUR.MINEUR.CORRECTIF** (ex: 3.12.7)

Incrément	Quand ?	Exemple
MAJEUR	Changement cassant (breaking change)	2.0.0 → 3.0.0
MINEUR	Nouvelle fonctionnalité, compatible	3.1.0 → 3.2.0
CORRECTIF	Bug fix, compatible	3.2.1 → 3.2.2

Règles importantes :

- Versions **0.x** : tout est permis (pas encore stable)
- Après 1.0 : le numéro de version est un **contrat**
- Le point n'est **pas** un séparateur décimal : après 1.9 vient 1.10

<https://semver.org>

****Versionnement calendaire (calver)****

Alternative au semver, basée sur la **date** :

Projet	Format	Exemple
Ubuntu	YY.MM	24.04
pip	YY.MINOR	24.0
Chrome	MAJOR (incrémental rapide)	125

Combinaisons possibles : YYYY.MINOR.PATCH, YY.MM.MINOR

Quand utiliser calver ? Quand la compatibilité arrière est moins pertinente que la fraîcheur (distributions, navigateurs).

<https://calver.org>

****Release notes et Changelog****

Pourquoi ?

- Informer sur les changements de **compatibilité**
- Permettre aux utilisateurs de **découvrir** les nouveautés
- Savoir **quand** un changement a été introduit

Standard "Keep a Changelog" (<https://keepachangelog.com>) :

Catégorie	Description
Added	Nouvelles fonctionnalités
Changed	Modifications (impact compatibilité possible)
Deprecated	Fonctionnalités bientôt supprimées
Removed	Fonctionnalités supprimées
Fixed	Corrections de bugs
Security	Corrections de vulnérabilités

****GitHub Releases****

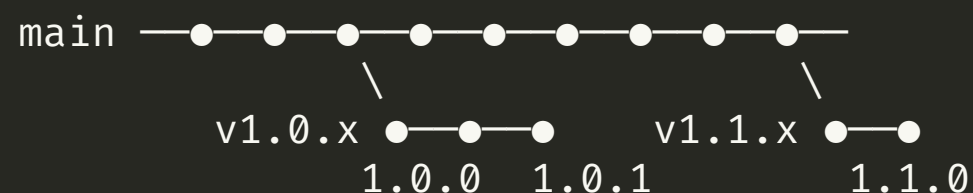
Fonctionnalités

- Création d'un **tag** Git
- **Notes de version** auto-générées (basées sur les PRs)
- Fichiers **attachés** (binaires, docs)
- Option version **préliminaire** (beta, RC)
- **Notification** aux watchers

Bonnes pratiques

- Utiliser en **complément** d'un **CHANGELOG.md**
- Créditer les **contributeurs**
- Catégoriser via les **labels** des PRs
- Automatiser avec **CI/CD** (GitHub Actions)

Branches et cycles de release



Deux approches pour le rythme :

Approche	Description	Exemple
Feature-based	On sort quand les features sont prêtes	Projets jeunes
Calendar-based	Cycle fixe (6 semaines, 6 mois ...)	Ubuntu, Chrome, Rust

Backporting : appliquer un correctif de `main` sur une branche stable.

****Résumé****

****Ce qu'il faut retenir****

1. **PRs** : Petites, bien décrites, testées
2. **Commits** : Messages clairs, format Conventional Commits
3. **Communication** : Async, polie, complète
4. **Review** : Contribuer à la review, pas seulement la recevoir
5. **Lancer un projet** : Ouvrir tôt, documenter progressivement, choisir des outils standards
6. **Releases** : Semver, changelog, cycles prévisibles

****Checklist avant de soumettre une PR****

- ☐ La PR ne contient qu'un changement logique
- ☐ Les tests passent localement
- ☐ Le code respecte les guidelines du projet
- ☐ Le message de commit est clair
- ☐ La description explique le pourquoi
- ☐ L'issue liée est référencée
- ☐ J'ai relu mon propre diff

****Pour la prochaine séance******Séance 8** : Gouvernance des projets open source**Préparation suggérée :**

- Regarder comment sont prises les décisions dans un projet que vous suivez
- Qu'est-ce qu'un BDFL ?

****Questions ?****

:::

Passons au TD